



ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using Pytorch

Year 2026 Term 1

Group # 1

Group Members:

#	Member Name	Member Academic ID
1	Khalid Waleed Alhilal (Group Leader)	2230000788
2	Ali Khalid Albinali	2230003813
3	Mishary Waleed AlAwadh	2230006217
4	Nawaf Alsabr	2230004159
5	Yasir Saleh Alghamdi	2230004157
6	Ibrahim Alkuwaiz	2230002878
7	Meshal Alrubaie	2230005991

GitHub repository: <https://github.com/KhalidExe/ARTI502-Assignment1>

Overview

In this assignment, you will perform Training Neural Net on the MNIST Dataset dataset using Pytorch. The goal is to explore the Basic Neural Network, understand its structure, and identify important issues such as architecture, learning rate, momentum and other parameters. You will also analyze relationships between number of layers and number of neurons for each layers to the overall Neural Network performance through their tuning hyperparameters.

Objective

Explore the dataset using PyTorch to understand:

- Simple Neural Networks structure
- How to do Training-Testing Neural Networks
- How to evaluate the performance
- How to determine learning rate, momentum and model hyperparameters

Summary of our results

All numbers below come from the executed notebook Assignment-1 DL.ipynb (NVIDIA GeForce RTX 5070 Ti, fixed seed 42). Hyper-parameters were chosen on a **validation** split only; the test set was used only for reporting.

Model	Val acc	Test loss	Test acc	Test errors
Task 6 – baseline MLP 784-512-10 (lr 0.01, momentum 0.9)	97.92 %	0.0603	98.26 %	174
Task 8 – tuned MLP 1×512 (lr 0.1, momentum 0.5)	98.26 %	0.0682	98.55 %	145
Extra – CNN + BatchNorm + Dropout + augmentation	99.67 %	0.0087	99.65 %	35

Source code: <https://github.com/KhalidExe/ARTI502-Assignment1>

Environment: Windows 11, Python 3.14.4, PyTorch 2.14.0+cu130 (CUDA 13.0), VS Code; AMD Ryzen 9 9950X, NVIDIA GeForce RTX 5070 Ti 16 GB, 64 GB DDR5.



Tasks (with PyTorch)

Task 1: Install PyTorch

Instructions:

- Install PyTorch and load it to your python code and save it as ‘Assignment-1 DL’
- Screenshoot how your load PyTorch to your python source code.

Our Work & Results

PyTorch was installed in a project virtual environment (.venv, Python 3.14.4) from the official PyTorch wheel index. The RTX 50-series (Blackwell) GPU needs a CUDA 12.8-or-newer build; the latest stable release, **PyTorch 2.14.0**, is published for **CUDA 13.0** (torch==2.14.0+cu130, <https://download.pytorch.org/whl/cu130>). The source code is saved as **Assignment-1 DL.ipynb** (plus a plain-Python export Assignment-1 DL.py).

```
PyTorch is installed in a project virtual environment (.venv, Python 3.14) from the official PyTorch wheel index. The RTX 50-series (Blackwell) GPU needs a CUDA 12.8-or-newer build; the latest stable release (torch 2.14.0) is published for CUDA 13.0, so requirements.txt pins torch=2.14.0+cu130 from https://download.pytorch.org/whl/cu130. Equivalent command:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu130

The first cell installs everything from requirements.txt (already-installed packages are skipped). The next cell loads PyTorch and the related libraries, prints their versions, verifies the GPU, and fixes all random seeds so every result in this notebook is reproducible.

D:\> pip install -q -r requirements.txt

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 26.0.1 -> 26.7.1
[notice] To update, run: python.exe -m pip install --upgrade pip

D:\>

import os
os.environ.setdefault("CUDA_VISIBLE_DEVICES", "-1") # needed for deterministic CUDA

import math, time, random, platform, sys, copy
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split

print("Python : {sys.version.split()[0]}")
print("PyTorch : {torch.__version__}")
print("torchvision : {torchvision.__version__}")
print("numpy : {np.__version__}")
print("pandas : {pd.__version__}")
print("matplotlib : {matplotlib.__version__}")
print("CUDA build : {torch.version.cuda}")
print("CUDA ready : {torch.cuda.is_available()}")

Python : 3.14.4
PyTorch : 2.14.0+cu130
torchvision : 0.20.0+cu130
numpy : 2.5.2
pandas : 3.0.4
matplotlib : 3.11.2
CUDA build : 13.0
CUDA ready : True

# GPU verification
assert torch.cuda.is_available(), "CUDA GPU not available - stop here"
DEVICE = torch.device("cuda")
print("New name : ", torch.cuda.get_device_name(0))
print("Compute cap. : ", torch.cuda.get_device_capability(0))
a = torch.randn(1000, 1000, device=DEVICE)
b = torch.randn(1000, 1000, device=DEVICE)
```

Figure 1: Loading PyTorch and the related libraries in Assignment-1 DL.ipynb (VS Code) – versions and CUDA availability



```

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
import random
import os
import sys
import time
import numpy as np
import math

# GPU verification
assert torch.cuda.is_available(), "GPU not available - stop here"
DEVICE = torch.device("cuda")
print("GPU name: ", torch.cuda.get_device_name(0))
print("Compute cap: ", torch.cuda.get_device_capability(0))
n = torch.randn(1000, 1000, device=DEVICE)
t = torch.randn(1000, 1000, device=DEVICE)
c = n @ t
torch.cuda.synchronize()
print("Tensor up on GPU", c.shape, " device:", c.device, " checksum:", round(c.sum().item(), 3))

# ... reproducibility ...
SEED = 42
def set_seed(seed=SEED):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
set_seed()
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
torch.use_deterministic_algorithms(True, warn_only=True)

# ... output folders + helpers ...
def save_fig(name):
    path = os.path.join("screenshots", name)
    plt.savefig(path, dpi=200, bbox_inches='tight')
    print("save -> ", path)

TIMINGS = [] # (model, device, epochs, seconds) - filled by every training run
print("seed: ", SEED, " deterministic cudnn: ", torch.backends.cudnn.deterministic)

GPU name      : NVIDIA GeForce RTX 5070 Ti
Compute cap   : (12, 0)
Tensor up on GPU: torch.Size([1000, 1000]) | device: cuda:0 | checksum: -34264.48
Seed: 42 | deterministic cudnn: True

Result:
PyTorch 2.14.0+cu130 (CUDA 12.0 build) and torchvision 0.29.0 are loaded in Python 3.14.4. torch.cuda.is_available() is True, the GPU is detected as NVIDIA GeForce RTX 5070 Ti with compute capability 12.0 (Blackwell), and a 1000x1000 matrix multiplication runs on cuda:0 - so the CUDA build works on this GPU. With seed 42 and deterministic cudnn every run is reproducible (for example, the 1x512 configuration gives exactly the same validation result in task 1a and task 1b).

Task 2: Download and Load the Dataset

torchvision.datasets.MNIST downloads the dataset from code into ./data. The official 60,000 training images are split (with a fixed seed) into 50,000 training images (the number stated in the assignment) and 10,000 validation images; the official 10,000 test images are kept untouched. Images are converted to tensors and normalised with the MNIST mean/std (0.1307, 0.3081).

mean, std = 0.1307, 0.3081
transform = transforms.Compose([
    transforms.ToTensor(), # uint8 [0,255] -> float [0,1], shape [1,28,28]
    transforms.Normalize((mean,), (std,)),
])

full_train = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_set = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_set, val_set = random_split(full_train, [50_000, 10_000],
                                  [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], generator=torch.Generator().manual_seed(42))

```

Figure 2: GPU verification and reproducibility settings

Result: PyTorch 2.14.0+cu130 and torchvision 0.29.0 are loaded, torch.cuda.is_available() returns **True**, the GPU is detected as **NVIDIA GeForce RTX 5070 Ti** (compute capability 12.0) and a 1000×1000 matrix multiplication runs on cuda:0. All random seeds (Python, NumPy, PyTorch) are fixed to 42 with deterministic cudnn, so every result in this report is reproducible – the notebook was run twice and gave identical accuracy and loss values.

Task 2: Download and Load the Dataset

Instructions:

- PyTorch has some built tools for downloading and loading common datasets. You will be playing around with MNIST in this assignment. It is a dataset of 28x28 grayscale handwritten digits 0-9. There are 50,000 images in the training set and 10,000 in the test set.
- Download the dataset directly from your python code then load it into the the workspace.

Deliverable:

- Screenshot showing the dataset loaded successfully.



Our Work & Results

```
torchvision.datasets.MNIST downloads the dataset from code into ./data. The official 60,000 training images are split (with a fixed seed) into 50,000 training images (the number stated in the assignment) and 10,000 validation images; the official 10,000 test images are kept untouched. Images are converted to tensors and normalised with the MNIST mean/std (0.1307, 0.3081).
```

```

MEAN, STD = 0.1307, 0.3081
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize([MEAN], [STD]),
])

full_train = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_set = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_set, val_set = random_split(full_train, [50_000, 10_000],
                                  generator=torch.Generator().manual_seed(42))

print("Full official training set : {}".format(len(full_train)))
print("  -> training subset       : {}".format(len(train_set)))
print("  -> validation subset      : {}".format(len(val_set)))
print("Official test set          : {}".format(len(test_set)))
print("Classes: ", full_train.classes)

```

```

Full official training set : 60000
-> training subset       : 50000
-> validation subset      : 10000
Official test set       : 10000
Classes: ['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six', '7 - seven', '8 - eight', '9 - nine']

```

Result:
MNIST was downloaded from code into ./data. The 60,000 official training images were split into 50,000 training and 10,000 validation images (fixed seed), and the 10,000 official test images are kept separate for the final evaluation. There are 10 classes (digits 0-9).

Task 3: Inspect the Dataset

A standard `DataLoader` with a mini-batch size of 32 is used to see how images are loaded into tensors.

```

BATCH_SIZE = 32
train_loader_std = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,
                              | 1 | 1 | 1 | 1 | generator=torch.Generator().manual_seed(42))
                              | 1 | 1 | 1 | 1 |
                              images, labels = next(iter(train_loader_std))

print("Images tensor size :", images.size())
print("Labels tensor size :", labels.size())
print("type / value range :", images.dtype, "[[images.min()...3], [images.max()...3]]")
print("Batches per epoch :", len(train_loader_std))

```

```

1) Images tensor size : torch.Size([32, 1, 28, 28])
Labels tensor size : torch.Size([32])
type / value range : torch.float32 [-0.434, 2.031]
Batches per epoch : 1863

```

Result:

Figure 3: MNIST downloaded from code with torchvision and split into 50,000 / 10,000 / 10,000

Result: `torchvision.datasets.MNIST(download=True)` downloads the dataset into `./data`. The official 60,000 training images are split (fixed seed) into **50,000 training** images – the number given in the assignment – and **10,000 validation** images used for tuning in Task 8. The official **10,000 test** images are kept separate for testing. Images are converted to tensors and normalised with the MNIST mean/std (0.1307, 0.3081).

Task 3: Inspect the Dataset

Instructions:

- It's worthwhile to check out how the dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also you can use matplotlib to see the visualization of the images data

Deliverable:

- Provide screenshots how to find:
 - Images tensor size
 - Data images vizualization using matplotlib

Our Work & Results

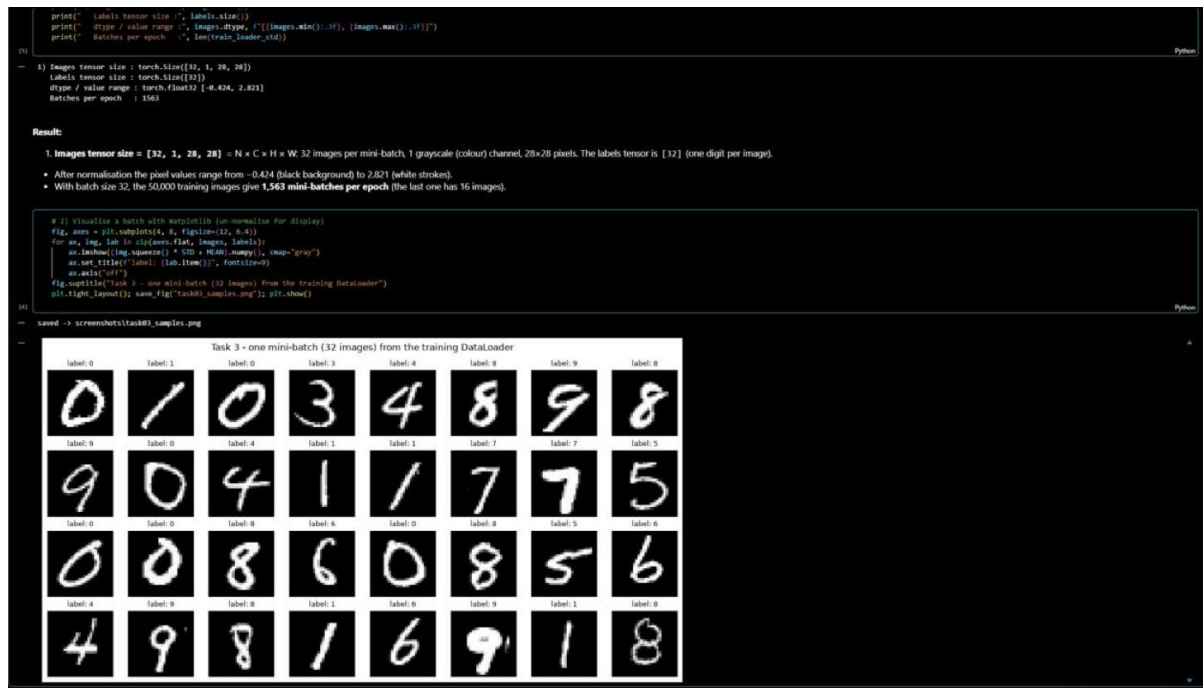


Figure 4: (1) images tensor size from the DataLoader and (2) visualisation of a mini-batch with matplotlib

1. Images tensor size = [32, 1, 28, 28] = $N \times C \times H \times W$: 32 images per mini-batch, 1 grayscale (colour) channel, 28 × 28 pixels; the labels tensor is [32]. After normalisation the pixel values range from -0.424 (background) to 2.821 (strokes). With batch size 32 the 50,000 training images give **1,563 mini-batches per epoch**.

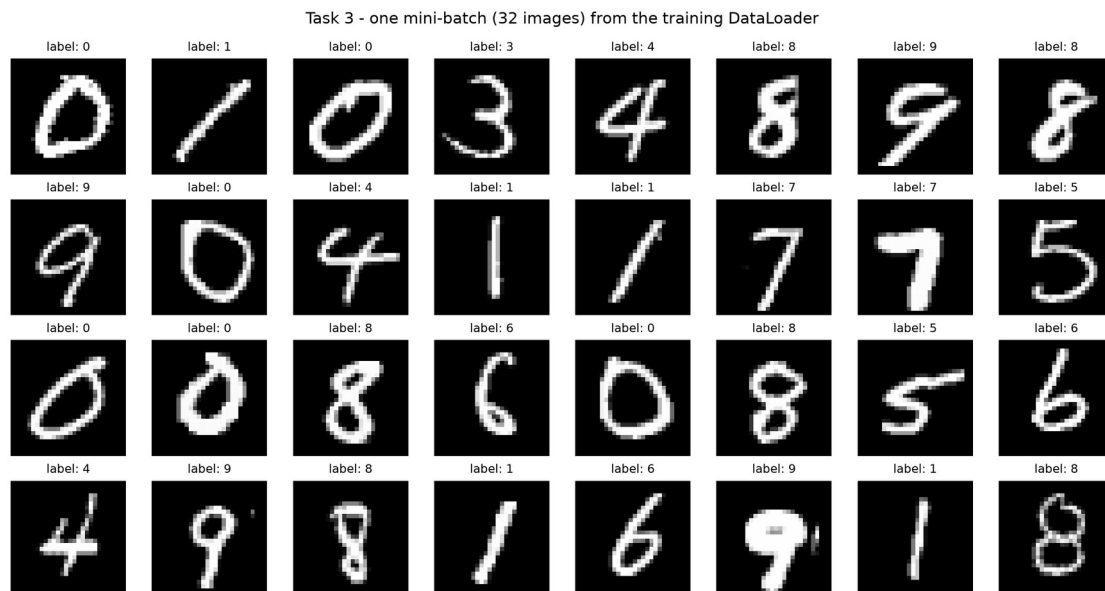


Figure 5: Task 3 – one mini-batch (32 images) from the training DataLoader with their labels

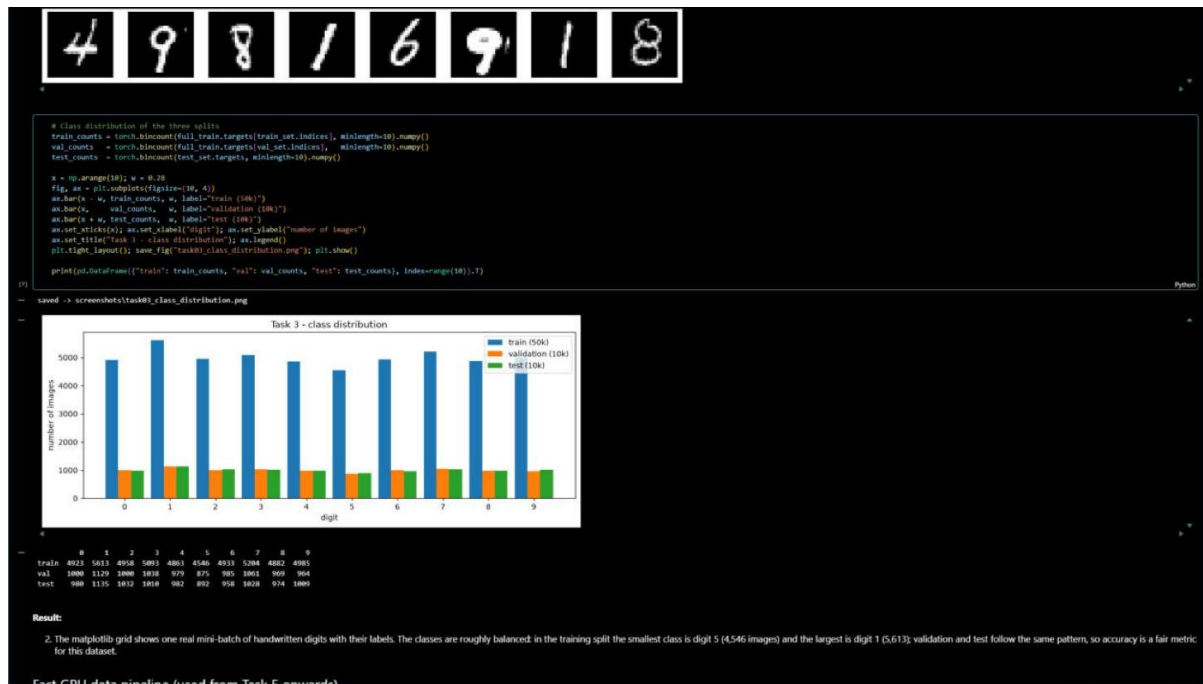


Figure 6: Class distribution of the training, validation and test splits

2. Visualisation: the matplotlib grid shows real handwritten digits with their labels. The class distribution shows the dataset is roughly balanced (training split: smallest class digit 5 with 4,546 images, largest digit 1 with 5,613), so accuracy is a fair metric. For training, all splits are then kept on the GPU once (same data and normalisation) to avoid per-batch copies.

Task 4: Creating A Simple Network

Instructions:

- Write your Python code to setup the following simple network architecture:

First you'll build a very basic neural network with a single hidden layer of neurons. This means one fully connected layer of weights connects the input to the hidden neurons and one fully connected layer connects the hidden neurons to the output. Use the RELU activation function on the hidden neuron values as our nonlinearity.

These fully connected (or `nn.Linear`) layers expect a 2D input tensor where that is $N \times I$ where N is the number of data points in a **mini batch** and I is the **number of inputs**. However, our data is formatted in $(N \times C \times H \times W)$ right now so we need to tell PyTorch to rearrange it using `torch.flatten`. C , H and W are refer to Color, Height and Width respectively.

After being **flattened** our data goes from $(32 \times 1 \times 28 \times 28)$ to being (32×784)

Deliverables:

- Screenshot of source code
- Verify your code with above requirement.



Our Work & Results

```
-- GPU tensors: (50000, 1, 28, 28) (10000, 1, 28, 28) (10000, 1, 28, 28) on cuda:0
Train batches per epoch: 1563

Task 4: Creating a Simple Network

One fully connected layer maps the 784 inputs to the hidden neurons, ReLU is applied to the hidden values, and a second fully connected layer maps the hidden neurons to the 10 output classes

input (N×1×28×28) → torch.flatten → (N×784) → Linear(784→512) → ReLU → Linear(512→10) → logits (N×10)

nn.Linear expects a 2-D input N × I, so torch.flatten(x, 1) keeps the batch dimension N and merges C×H×W = 1×28×28 = 784.

class SimpleNet(nn.Module):
    def __init__(self, inputs=28 * 28, hidden=512, outputs=10):
        super().__init__()
        self.fc1 = nn.Linear(inputs, hidden) # input → hidden
        self.fc2 = nn.Linear(hidden, outputs) # hidden → output

    def forward(self, x):
        x = torch.flatten(x, 1) # (N,1,28,28) → (N,784)
        x = F.relu(self.fc1(x)) # ReLU non-linearity on hidden neurons
        x = self.fc2(x) # raw scores (logits), softmax is inside CrossEntropyLoss
        return x

def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

net = SimpleNet()
print(net)
print(f"Trainable parameters: {count_params(net):,} (784*512+512 + 512*10+10)")

-- SimpleNet(
  (fc1): Linear(in_features=784, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=10, bias=True)
)
Trainable parameters: 407,850 (784*512+512 + 512*10+10)

# verify the shapes with one real mini-batch from the task 3 Dataloader
print("batch before flatten :", tuple(images.shape))
flat = torch.flatten(images, 1)
print("batch after flatten :", tuple(flat.shape))
with torch.no_grad():
    hidden = F.relu(net.fc1(flat))
    out = net(images)
print("hidden layer output :", tuple(hidden.shape))
print("network output :", tuple(out.shape))
score: flat.shape == (32, 784) and out.shape == (32, 10)
print("Shape check passed: (32 x 1 x 28 x 28) → (32 x 784) → (32 x 10)")

-- batch before flatten : (32, 1, 28, 28)
batch after flatten : (32, 784)
hidden layer output : (32, 512)
network output : (32, 10)
Shape check passed: (32 x 1 x 28 x 28) → (32 x 784) → (32 x 10)
```

Figure 7: SimpleNet source code (Linear 784 → 512, ReLU, Linear 512 → 10 with torch.flatten) and shape verification

Architecture: input (N×1×28×28) → torch.flatten(x, 1) → (N×784) → Linear(784→512) → ReLU → Linear(512→10) → logits (N×10). The network has **407,050** trainable parameters (784·512 + 512 = 401,920 and 512·10 + 10 = 5,130).

Verification: running one real mini-batch through the network prints (32, 1, 28, 28) before flattening, **(32, 784) after torch.flatten**, (32, 512) after the hidden layer and (32, 10) at the output – exactly the required (32 × 1 × 28 × 28) → (32 × 784) transformation, with one score per class for every image.

Task 5: Training Function

Instructions:

- Now for training the network. Your train function takes as input the net to train and the dataloader for the training data. It also takes some optional parameters to control training.
- For your network you will be using PyTorch's built in nn.CrossEntropyLoss. This will apply a softmax to our network's output, calculate the log-probability assigned to each class, then try to minimize the negative log likelihood of our data (AKA maximize the likelihood)
- For your optimizer you are using stochastic gradient descent with learning rate and momentum

Deliverables:

- Screenshot of your python source code that contains your training function with above requirements



Our Work & Results

```
with torch.no_grad():
    hidden = nn.Linear(1, 1)
    out = net(images)
    print("hidden layer output: ", tuple(hidden.shape))
    print("network output: ", tuple(out.shape))
    assert flat.shape == (32, 784) and out.shape == (32, 10)
    print("shape check passed: (32 x 1 x 28 x 28) -> (32 x 784) -> (32 x 10)")

-- batch before flatten : (32, 1, 28, 28)
-- batch after flatten : (32, 784)
-- hidden layer output : (32, 512)
-- network output : (32, 10)
-- shape check passed: (32 x 1 x 28 x 28) -> (32 x 784) -> (32 x 10)

Result:
The network matches the required structure: fc1 784-512, ReLU, fc2 512-10, with 407,050 trainable parameters (784*512 + 512 * 401,920 and 512*10 + 10 = 5,130). Running a real batch verifies the shapes: (32 x 1 x 28 x 28) -> torch.flatten -> (32 x 784) -> hidden layer (32 x 512) -> output (32 x 10), i.e. one score (logit) per class for each image.

Task 5: Training Function

train(net, train_loader, ...) takes the network and the training data loader plus optional parameters (epochs, learning rate, momentum, weight decay, validation loader, print interval).

• Loss: nn.CrossEntropyLoss - applies log-softmax to the network outputs and minimises the negative log-likelihood of the correct class (i.e. maximises the likelihood).
• Optimizer: torch.optim.SGD with learning rate and momentum.
• Debug output: every print_every batches it prints epoch, batch number, current (running) loss and accuracy.
• Returns: the list of all mini-batch losses, plus a history dict with per-epoch train/validation loss & accuracy.

@torch.no_grad()
def evaluate(net, loader, return_preds=False):
    """Average loss and accuracy (%) of 'net' over 'loader'."""
    net.eval()
    crit = nn.CrossEntropyLoss(reduction='sum')
    loss_sum, correct, n_preds = 0.0, 0, 0
    for xb, yb in loader:
        out = net(xb)
        loss_sum += crit(out, yb).item()
        n += out.argmax().item()
        correct += (n == yb).sum().item()
    if return_preds:
        preds.append(out)
    res = (loss_sum / n, 100.0 * correct / n)
    return res + (torch.cat(preds),) if return_preds else res

def train(net, train_loader, epochs=1, lr=0.01, momentum=0.9, weight_decay=0.0,
         val_loader=None, print_every=20, verbose=True, name="model"):
    net.train()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum, weight_decay=weight_decay)
    losses = []
    history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}
    n = time.perf_counter()
    for epoch in range(epochs):
        net.train()
        ep_losses, ep_correct, ep_n = [], torch.zeros(1), device=DEVICE, 0
        for batch_idx, (xb, yb) in enumerate(train_loader):
            optimizer.zero_grad()
            out = net(xb)
            loss = criterion(out, yb)
            loss.backward()
            optimizer.step()

            correct = (out.argmax(1) == yb).sum()
            ep_losses.append(loss.detach()); ep_correct += correct; ep_n += yb.numel()
            if verbose and (1 + 3) % print_every == 0:
                recent = torch.stack(ep_losses[-print_every:]).mean().item()
                acc = 100.0 * ep_correct.item() / ep_n
                print(f"epoch {epoch+1:3} | epoch | batch {1+3:5} | (len(train_loader)) "
                      f"| {recent:4.1f} | running acc {acc:6.2f}%")
            ep_losses.append(loss.detach())
        ep_losses, ep_correct, ep_n = torch.stack(ep_losses).cpu(), losses.extend(ep_losses.tolist())
        tr_loss, tr_acc = ep_losses.mean().item(), 100.0 * ep_correct.item() / ep_n
        history["train_loss"].append(tr_loss); history["train_acc"].append(tr_acc)
        if val_loader is not None:
            val_loss, val_acc = evaluate(net, val_loader)
            history["val_loss"].append(val_loss); history["val_acc"].append(val_acc)
        if verbose:
            msg = f"==> epoch {epoch+1:2} done | train loss {tr_loss:4f} | acc {tr_acc:6.2f}%"
            if val_loader is not None:
                msg += f" | val loss {val_loss:4f} | acc {val_acc:6.2f}%"
            print(msg)
            if not math.isfinite(tr_loss):
                # diverged (too large lr) -> stop early
                if verbose: print("loss diverged - stopping")
                break
        torch.cuda.synchronize()
        history["time_s"] = time.perf_counter() - n
        tqdm.tqdm.write(f"Model: {name}, device: {str(DEVICE)}, epochs: {epoch}, seconds: {history['time_s']}")
        return losses, history

print("train() and evaluate() defined")

-- train() and evaluate() defined
```

Figure 8: evaluate() and the start of train(net, train_loader, epochs, lr, momentum, ...) – nn.CrossEntropyLoss and optim.SGD(lr, momentum)

```
optimizer.zero_grad(set_to_none=True)
out = net(xb)
loss = criterion(out, yb)
loss.backward()
optimizer.step()

correct = (out.argmax(1) == yb).sum()
ep_losses.append(loss.detach()); ep_correct += correct; ep_n += yb.numel()
if verbose and (1 + 3) % print_every == 0:
    recent = torch.stack(ep_losses[-print_every:]).mean().item()
    acc = 100.0 * ep_correct.item() / ep_n
    print(f"epoch {epoch+1:3} | epoch | batch {1+3:5} | (len(train_loader)) "
          f"| {recent:4.1f} | running acc {acc:6.2f}%")
ep_losses.append(loss.detach())
ep_losses, ep_correct, ep_n = torch.stack(ep_losses).cpu(), losses.extend(ep_losses.tolist())
tr_loss, tr_acc = ep_losses.mean().item(), 100.0 * ep_correct.item() / ep_n
history["train_loss"].append(tr_loss); history["train_acc"].append(tr_acc)
if val_loader is not None:
    val_loss, val_acc = evaluate(net, val_loader)
    history["val_loss"].append(val_loss); history["val_acc"].append(val_acc)
if verbose:
    msg = f"==> epoch {epoch+1:2} done | train loss {tr_loss:4f} | acc {tr_acc:6.2f}%"
    if val_loader is not None:
        msg += f" | val loss {val_loss:4f} | acc {val_acc:6.2f}%"
    print(msg)
    if not math.isfinite(tr_loss):
        # diverged (too large lr) -> stop early
        if verbose: print("loss diverged - stopping")
        break
torch.cuda.synchronize()
history["time_s"] = time.perf_counter() - n
tqdm.tqdm.write(f"Model: {name}, device: {str(DEVICE)}, epochs: {epoch}, seconds: {history['time_s']}")
return losses, history

print("train() and evaluate() defined")

-- train() and evaluate() defined

Task 6: Training the Network

A new SimpleNet (784 -> 512 -> 10) is instantiated and trained on the 50,000 training images with mini-batch size 32, 10 epochs, learning rate 0.01, momentum 0.9. The validation split is evaluated after every epoch.

EPOCHS, LR, MOMENTUM = 10, 0.01, 0.9
net = SimpleNet()
train_loader, val_loader, test_loader = make_loaders()
losses, hist = train(net, train_loader, epochs=EPOCHS, lr=LR, momentum=MOMENTUM,
                    val_loader=val_loader, print_every=20, name="SimpleNet baseline (Task 6)")
print(f"Training time: {hist['time_s']:.1f} s | mini-batch losses returned: {len(losses)}")

-- epoch 1/10 | batch 250/1563 | loss 0.4023 | running acc: 84.91%
-- epoch 1/10 | batch 500/1563 | loss 0.2600 | running acc: 88.30%
-- epoch 1/10 | batch 750/1563 | loss 0.2040 | running acc: 90.08%
-- epoch 1/10 | batch 1000/1563 | loss 0.1529 | running acc: 91.81%
-- epoch 1/10 | batch 1250/1563 | loss 0.1150 | running acc: 92.40%
-- epoch 1/10 | batch 1500/1563 | loss 0.1102 | running acc: 93.10%
--> epoch 1 done | train loss 0.2107 | acc: 91.34% | val loss 0.1205 | acc: 96.44%
-- epoch 2/10 | batch 250/1563 | loss 0.0988 | running acc: 97.51%
```

Figure 9: Rest of the training loop – debug printing, per-epoch validation and return values

train(net, train_loader, epochs, lr, momentum, weight_decay, val_loader, print_every, ...) takes the network and the training data loader plus optional parameters. It uses **nn.CrossEntropyLoss** (log-softmax + negative log-likelihood) and **torch.optim.SGD with learning rate and momentum**. Each step does zero_grad → forward → loss → backward → optimizer.step. Every print_every batches it prints the epoch, batch number, current loss and running accuracy; after each epoch it evaluates the validation set. It returns **the list of all mini-batch losses** and a history of per-epoch train/validation loss and accuracy.

Task 6: Training Network

Instructions:

- You will instantiate a new network and train it on your training data.
- Your training function prints out some debug information about the epoch, batch number, and current loss and accuracy values. It also returns a list of all the losses on your mini-batches so you can plot them all once training has finished.

Deliverables:

- Screenshot of instantiate training source code
- Screenshot of training debug which include epoch, batch number and current loss, accuracy values and graph for overall processes (loss and accuracy for each epoch).

Our Work & Results

A new SimpleNet is instantiated and trained on the 50,000 training images with mini-batch size 32, 10 epochs, learning rate 0.01, momentum 0.9 (instantiation code at the bottom of Figure 9).

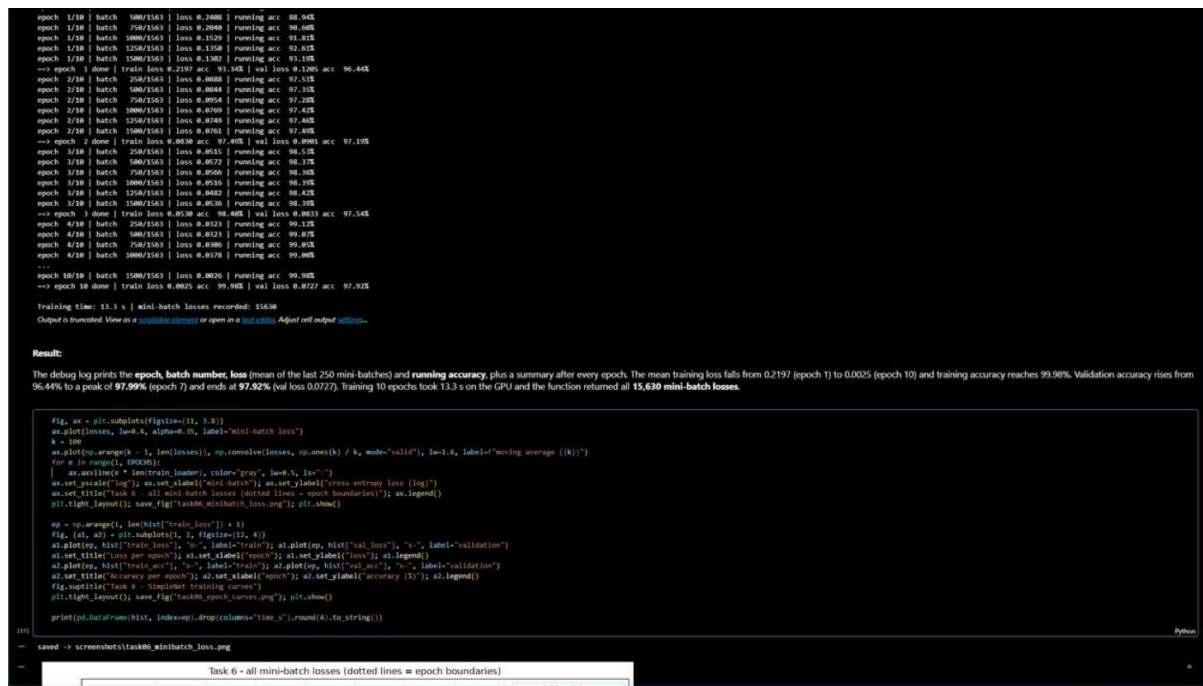


Figure 10: Training debug output – epoch, batch number, current loss and running accuracy, and per-epoch summary

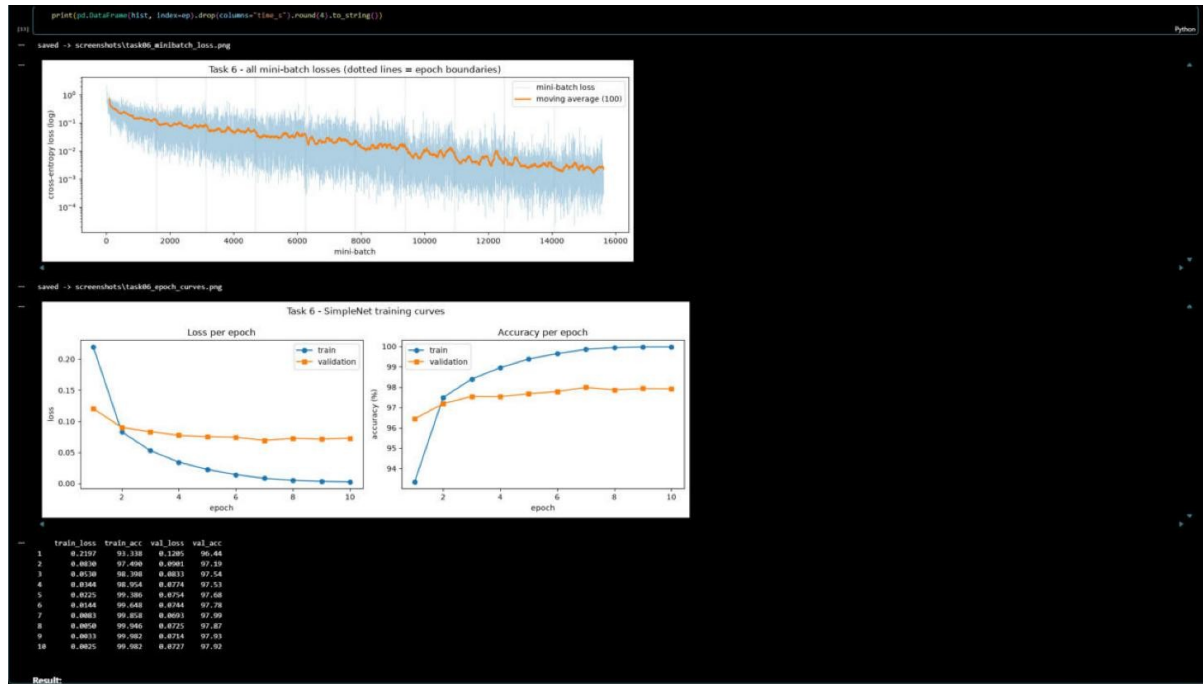


Figure 11: Graphs of all mini-batch losses and of loss / accuracy for each epoch

Epoch	Train loss	Train acc	Val loss	Val acc
1	0.2197	93.34 %	0.1205	96.44 %
2	0.0830	97.49 %	0.0901	97.19 %
3	0.0530	98.40 %	0.0833	97.54 %
4	0.0344	98.95 %	0.0774	97.53 %
5	0.0225	99.39 %	0.0754	97.68 %
6	0.0144	99.65 %	0.0744	97.78 %
7	0.0083	99.86 %	0.0693	97.99 %
8	0.0050	99.95 %	0.0725	97.87 %
9	0.0033	99.98 %	0.0714	97.93 %
10	0.0025	99.98 %	0.0727	97.92 %

Result: the debug log shows the epoch, batch number (out of 1,563), current loss and accuracy every 250 batches. The mean training loss falls from 0.2197 to **0.0025** and training accuracy reaches 99.98 %. Validation accuracy rises from 96.44 % to a peak of **97.99 %** at epoch 7 and ends at **97.92 %** (val loss 0.0727). The function returned all **15,630 mini-batch losses** ($10 \times 1,563$); training took 13.3 s on the GPU.

The mini-batch loss curve decreases steadily (log scale) with noise because every point is a single batch of 32 images. The per-epoch curves show that after about 7 epochs the validation loss stops improving while the training loss keeps going to zero – the ≈ 2 -point gap between training (99.98 %) and validation (97.92 %) accuracy is the beginning of **overfitting**.



Task 7: Testing Network

Instructions:

- You can test your network on a few of your test images from task 2 and see what happens.

Deliverables:

- Screenshot your network testing source code
- Screenshot the results of the network testing

Our Work & Results

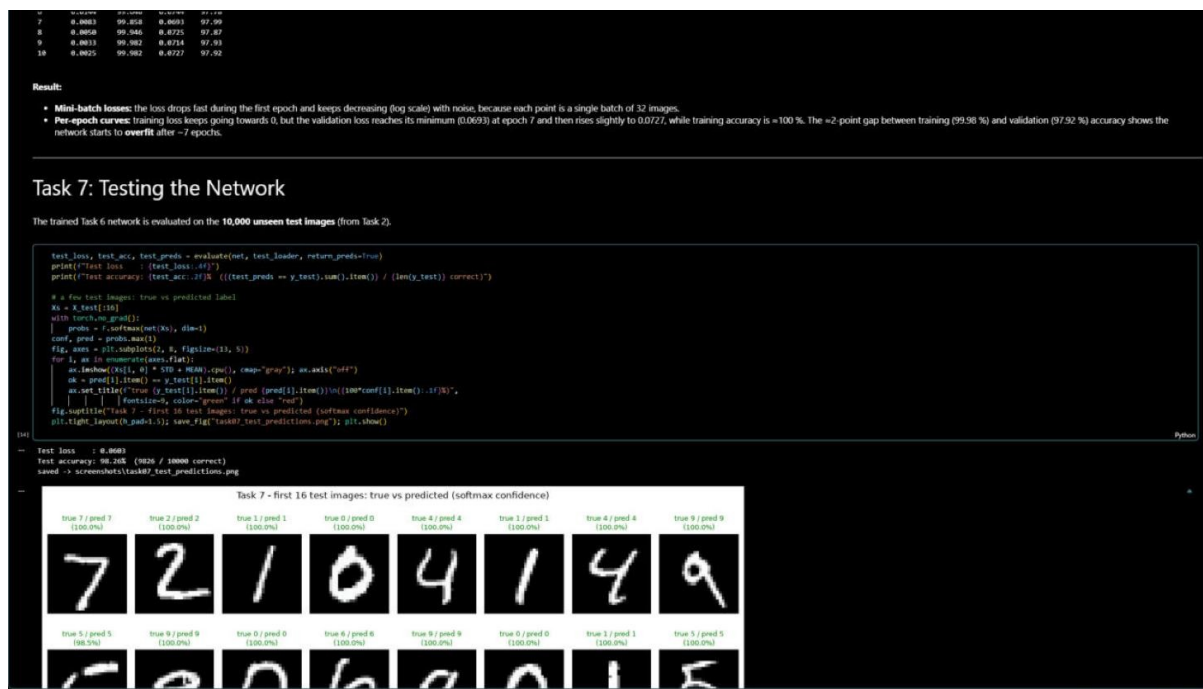


Figure 12: Network testing source code and results on the 10,000 test images

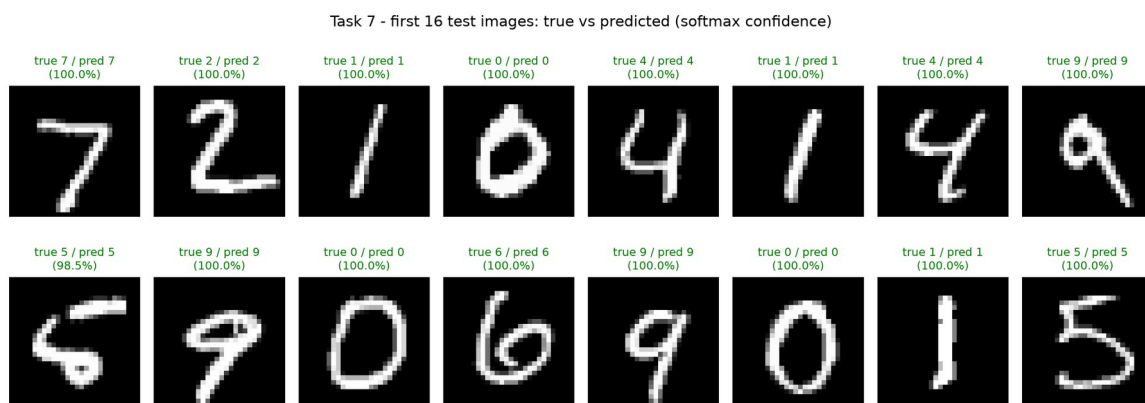


Figure 13: First 16 test images – true vs predicted label (softmax confidence)

Result: on the 10,000 unseen test images the Task 6 network reaches **test loss 0.0603** and **test accuracy 98.26 %** (9,826 correct). All 16 test images shown are predicted correctly; 15 with ≈100 % confidence and the slanted “5” with 98.5 %.

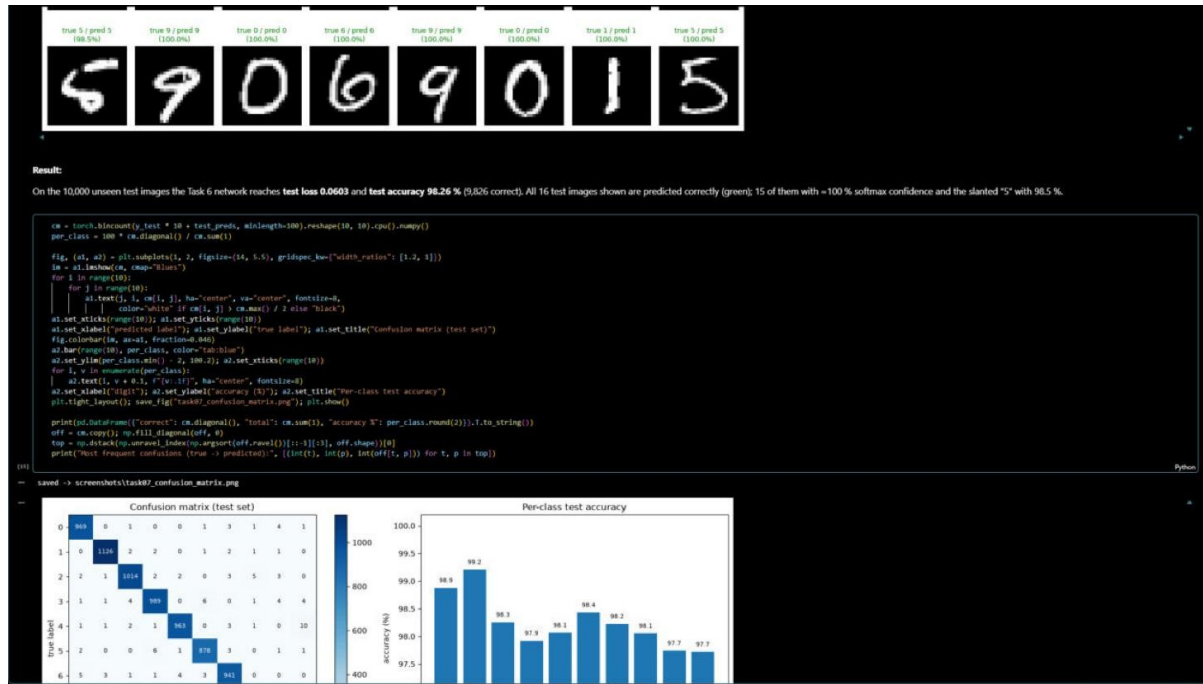


Figure 14: Confusion matrix and per-class accuracy – source code and output

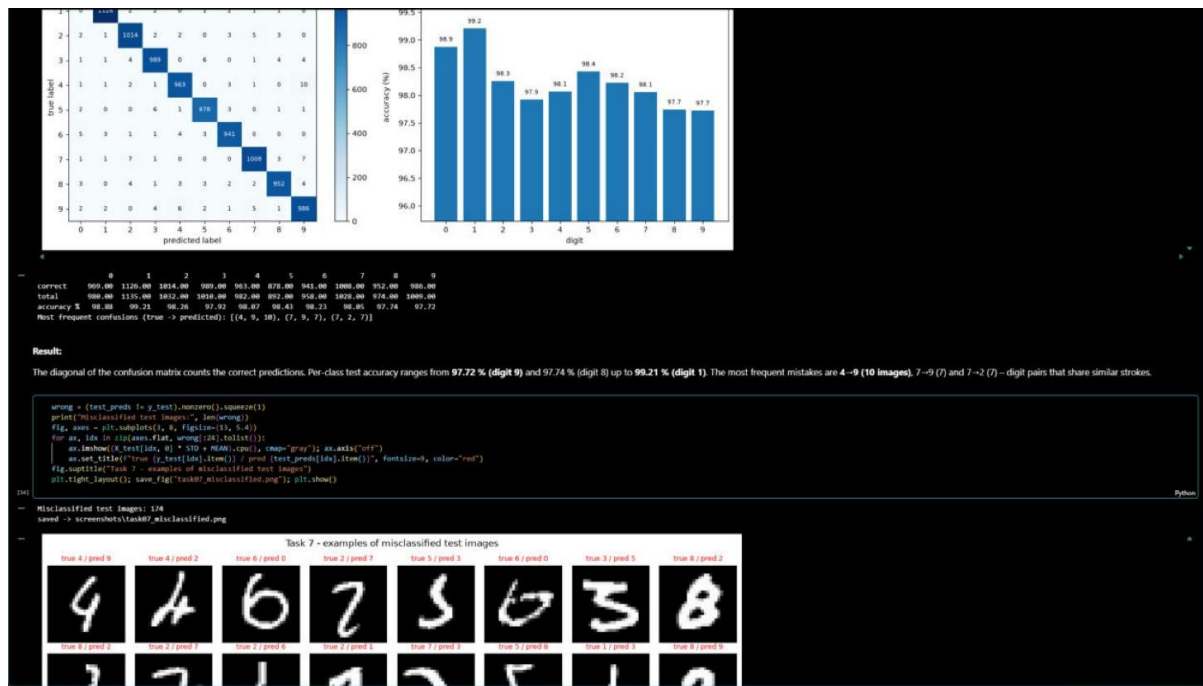


Figure 15: Per-class accuracy table, most frequent confusions and misclassified examples

Per-class test accuracy ranges from **97.72 % (digit 9)** and **97.74 % (digit 8)** to **99.21 % (digit 1)**. The most frequent mistakes are **4 → 9 (10 images)**, **7 → 9 (7)** and **7 → 2 (7)** – digits with similar strokes. In total **174** test images (1.74 %) are misclassified; most of them are unusual or badly written digits that are hard even for a human.



Task 7 - examples of misclassified test images

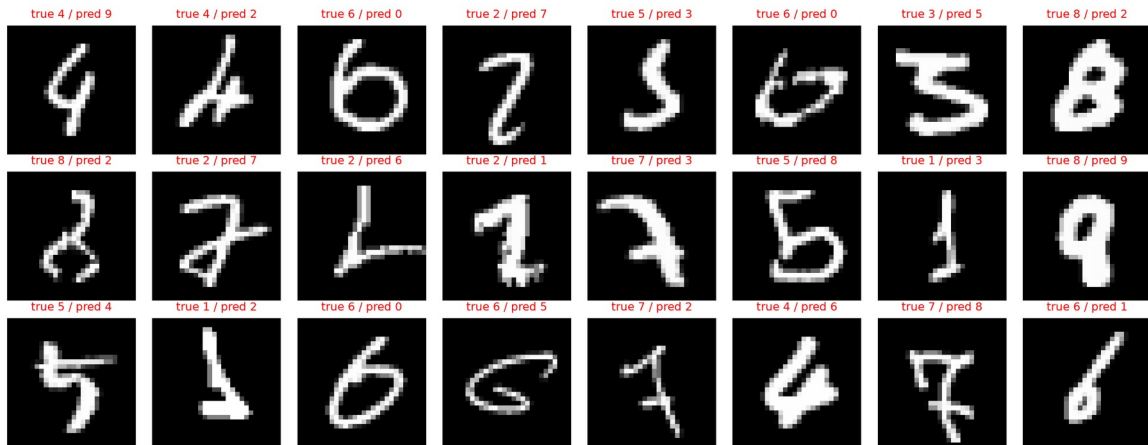


Figure 16: Examples of misclassified test images (true / predicted)

Task 8: Tuning the parameters and model architecture

Instructions:

- Tune the parameters with different values of learning rate and momentum and write your best results (lowest loss and highest accuracy).
- Tune the architecture model with different values of number of layer and number of neuron and write your best results (lowest loss and highest accuracy).

Deliverables:

- Screen your source python code with the best results.

Our Work & Results

Protocol: every configuration is trained on the 50,000 training images for 8 epochs with the same seed and compared on the **validation** set (highest validation accuracy, ties broken by lowest validation loss). The test set is not used for any choice; it is evaluated only once for the final model.

8a) Learning rate $\times$ momentum (SimpleNet 784-512-10)

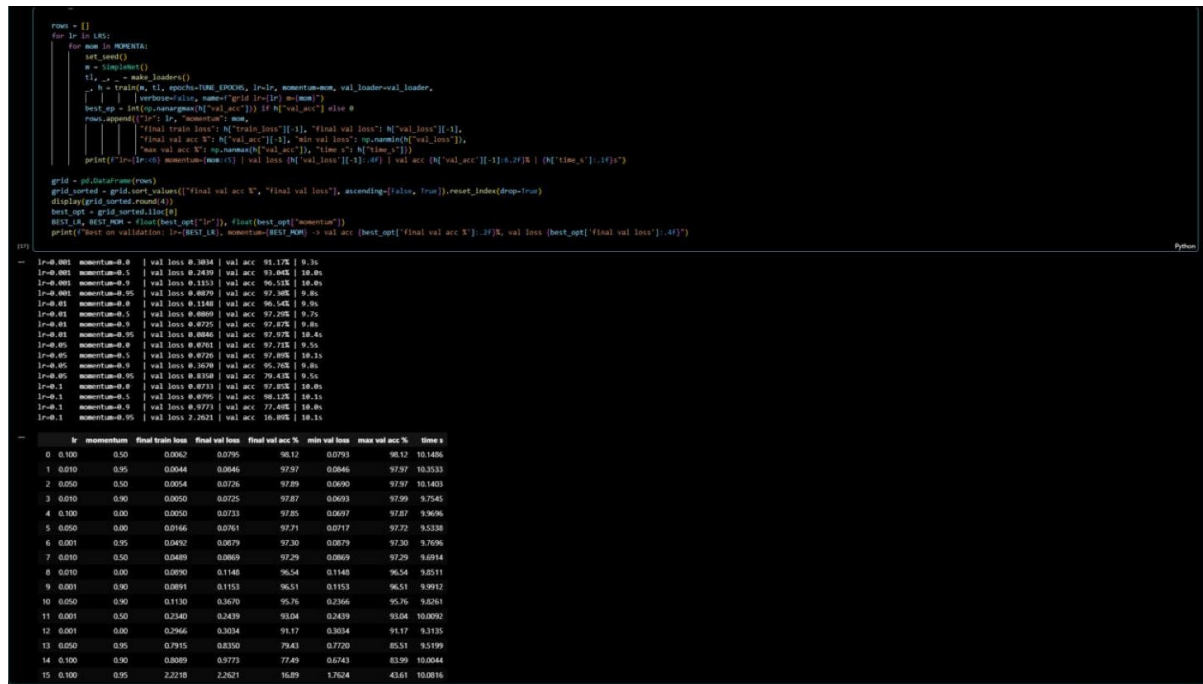


Figure 17: Grid search over learning rate $\{0.001, 0.01, 0.05, 0.1\} \times$ momentum $\{0, 0.5, 0.9, 0.95\}$ – source code and results

Task 8a - learning rate x momentum grid search (SimpleNet 784-512-10)

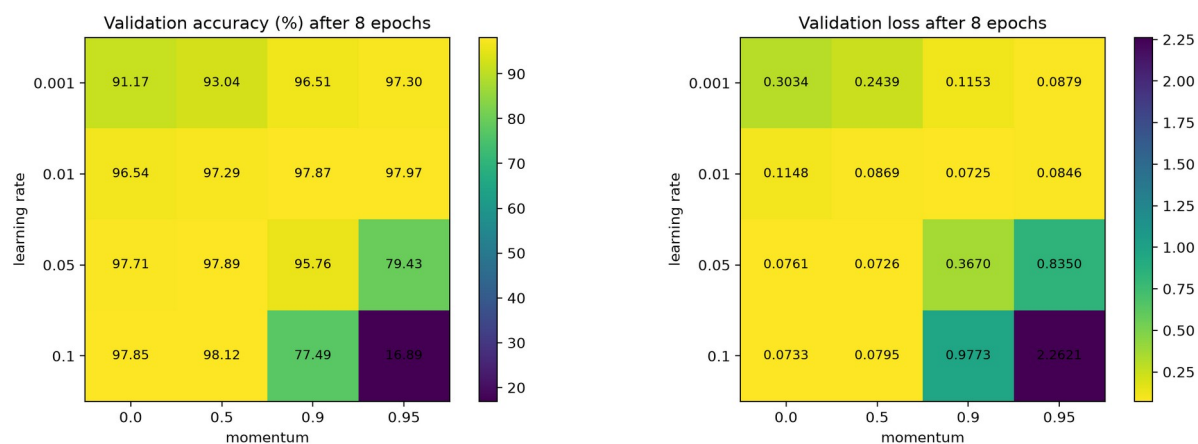


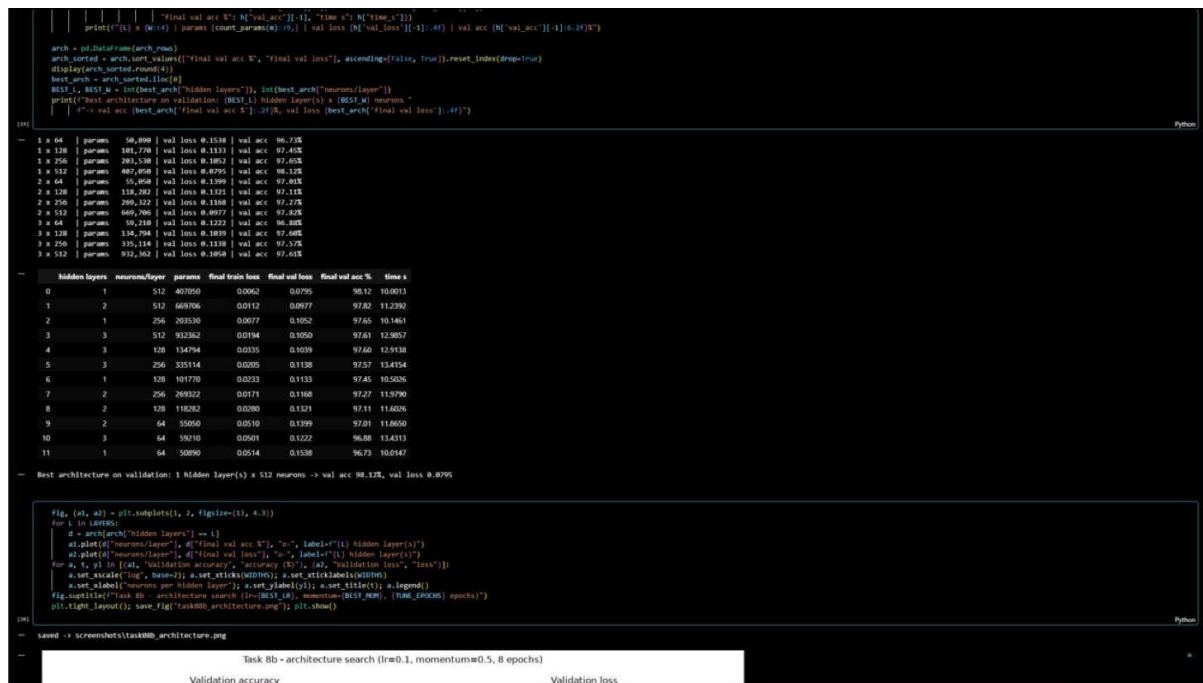
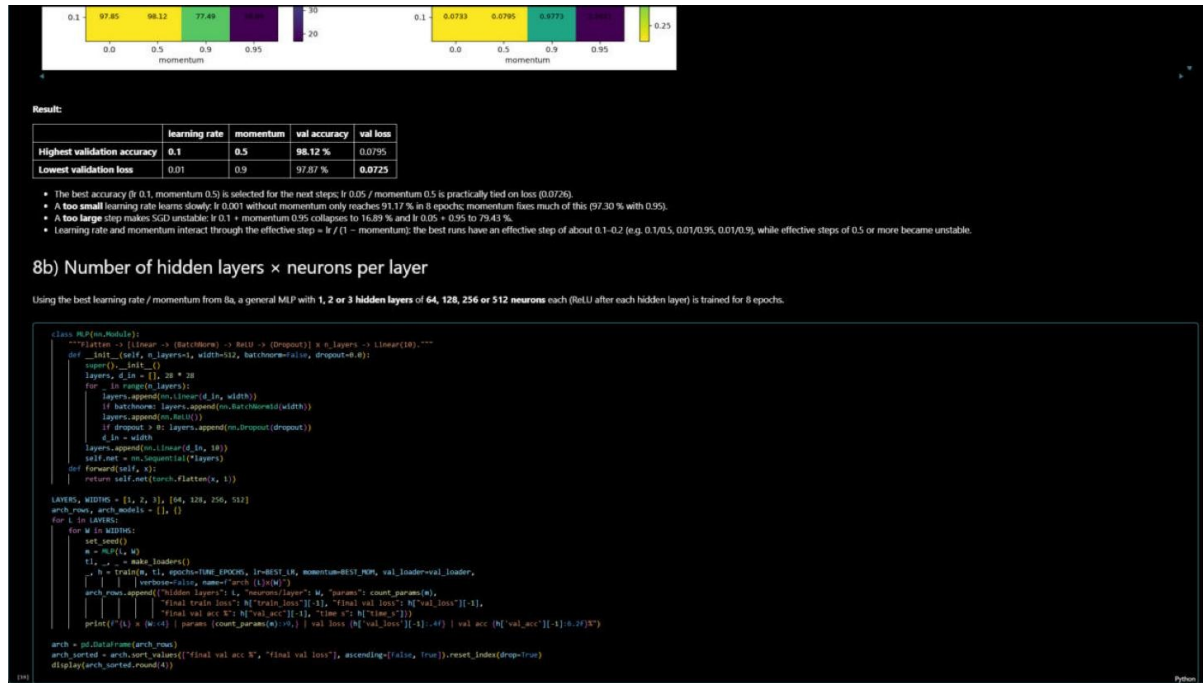
Figure 18: Validation accuracy and loss for every learning rate / momentum pair

Best result (validation)	Learning rate	Momentum	Val accuracy	Val loss
Highest accuracy	0.1	0.5	98.12 %	0.0795
Lowest loss	0.01	0.9	97.87 %	0.0725

- A **too small** learning rate learns slowly: lr 0.001 without momentum reaches only 91.17 % in 8 epochs; momentum helps a lot (97.30 % with 0.95).
- A **too large** step makes SGD unstable: lr 0.1 + momentum 0.95 collapses to 16.89 % and lr 0.05 + 0.95 to 79.43 %.

- Learning rate and momentum interact through the effective step $\approx \text{lr} / (1 - \text{momentum})$. The best runs have an effective step of about 0.1–0.2 (0.1/0.5, 0.01/0.95, 0.01/0.9); effective steps of 0.5 or more became unstable.

8b) Number of hidden layers $\times$ neurons per layer (lr 0.1, momentum 0.5)





Task 8b - architecture search (lr=0.1, momentum=0.5, 8 epochs)

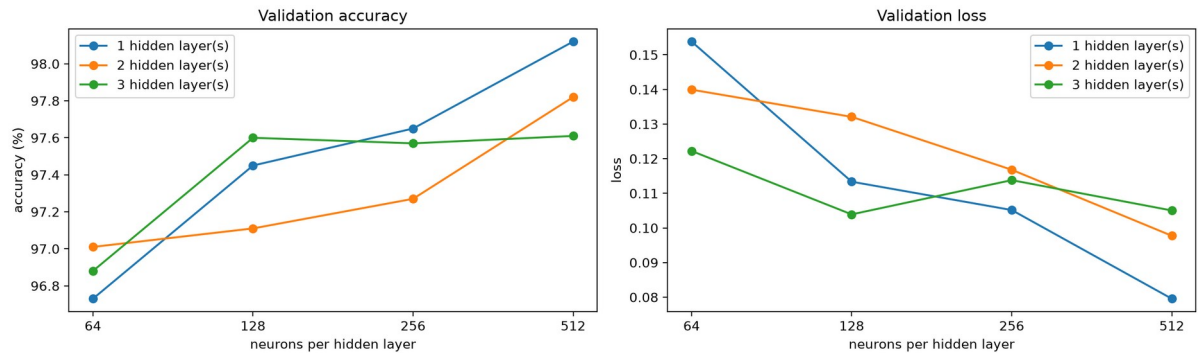


Figure 21: Validation accuracy and loss vs. neurons per hidden layer for 1, 2 and 3 hidden layers

Hidden layers × neurons	Params	Val accuracy	Val loss
1 × 512 (best: highest accuracy and lowest loss)	407,050	98.12 %	0.0795
2 × 512	669,706	97.82 %	0.0977
1 × 256	203,530	97.65 %	0.1052
3 × 512	932,362	97.61 %	0.1050
1 × 64 (worst)	50,890	96.73 %	0.1538

- **More neurons help** at every depth (1 hidden layer: 96.73 % with 64 → 98.12 % with 512 neurons).
- **More layers did not help here:** 3 × 512 reaches 97.61 % with 2.3× the parameters. With 8 epochs and a learning rate tuned for the one-layer network, deeper MLPs are harder to optimise, and MNIST is simple enough for one wide hidden layer.



8c) Final model with the best settings

```
def train_with_checkpoint(net, epochs, lr, momentum, name):
    """Runs a crossentropy training as train(), but keeps the weights of the best-validation epoch."""
    net.to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(net.parameters(), lr=lr, momentum=momentum)
    tr_loader, val_loader = make_loaders()
    hist = {"train_loss": [], "val_loss": [], "val_acc": []}
    best_key, best_state, best_epoch = None, None, None
    tb = time.perf_counter()
    for ep in range(epochs):
        net.train(); tr_loader.reset()
        for sb, yb in tr_loader:
            optimizer.zero_grad(set_to_none=True)
            loss = criterion(net(sb), yb); loss.backward(); optimizer.step()
            ls.append(loss.detach())
        tr_loss = torch.stack(ls).mean().item()
        v_loss, v_acc = evaluate(net, val_loader)
        hist["train_loss"].append(tr_loss); hist["val_loss"].append(v_loss); hist["val_acc"].append(v_acc)
        key = (v_acc, -v_loss)
        if best_key is None or key > best_key:
            best_key, best_state, best_epoch = key, copy.deepcopy(net.state_dict()), ep + 1
        print(f"Epoch {ep+1:2d}/{epochs} | train loss {tr_loss:.4f} | val loss {v_loss:.4f} | val acc {v_acc*100:.2f}%")
        torch.cuda.synchronize()
        hist["time_"] = time.perf_counter() - tb
        tbunch.append({"name": name, "device": str(DEVICE), "epoch": epochs, "seconds": hist["time_"]})
    net.load_state_dict(best_state)
    hist["best_epoch"] = best_epoch
    return hist

net.load()
final_net = MLP(BEST_L, BEST_W)
final_hist = train_with_checkpoint(final_net, FINAL_EPOCHS, BEST_LR, BEST_MOM, f"tuned MLP {BEST_L}x{BEST_W} (task 8c)")
# = final_hist["best_epoch"]
print(f"Selected epoch {best_epoch}: val loss {final_hist['val_loss'][best_epoch-1]:.4f}, val acc {final_hist['val_acc'][best_epoch-1]:.2f}%")
# f" training time {final_hist['time_'][best_epoch-1]:.2f}s")

epoch 1/20 | train loss 0.1904 | val loss 0.0801 | val acc 96.26%
epoch 2/20 | train loss 0.0836 | val loss 0.0499 | val acc 96.63%
epoch 3/20 | train loss 0.0538 | val loss 0.0982 | val acc 97.20%
epoch 4/20 | train loss 0.0129 | val loss 0.0793 | val acc 97.40%
epoch 5/20 | train loss 0.0219 | val loss 0.0924 | val acc 97.62%
epoch 6/20 | train loss 0.0142 | val loss 0.0888 | val acc 97.97%
epoch 7/20 | train loss 0.0072 | val loss 0.0881 | val acc 98.46%
epoch 8/20 | train loss 0.0062 | val loss 0.0795 | val acc 98.12%
epoch 9/20 | train loss 0.0033 | val loss 0.0794 | val acc 98.42%
epoch 10/20 | train loss 0.0034 | val loss 0.0773 | val acc 98.10%
epoch 11/20 | train loss 0.0004 | val loss 0.0778 | val acc 98.23%
epoch 12/20 | train loss 0.0001 | val loss 0.0777 | val acc 98.22%
epoch 13/20 | train loss 0.0002 | val loss 0.0757 | val acc 98.22%
epoch 14/20 | train loss 0.0002 | val loss 0.0788 | val acc 98.23%
epoch 15/20 | train loss 0.0002 | val loss 0.0793 | val acc 98.26%
epoch 16/20 | train loss 0.0002 | val loss 0.0795 | val acc 98.33%
epoch 17/20 | train loss 0.0002 | val loss 0.0800 | val acc 98.22%
epoch 18/20 | train loss 0.0002 | val loss 0.0801 | val acc 98.23%
epoch 19/20 | train loss 0.0001 | val loss 0.0806 | val acc 98.33%
epoch 20/20 | train loss 0.0001 | val loss 0.0809 | val acc 98.33%

Selected epoch 15: val loss 0.0793, val acc 98.26% | training time 22.6s

# = the DNN test-set evaluation of the tuned model
final_val_loss, final_val_acc = evaluate(final_net, val_loader)
final_test_loss, final_test_acc = evaluate(final_net, test_loader)
```

Figure 22: Final model 1 × 512, lr 0.1, momentum 0.5 trained for 20 epochs – best-validation epoch kept

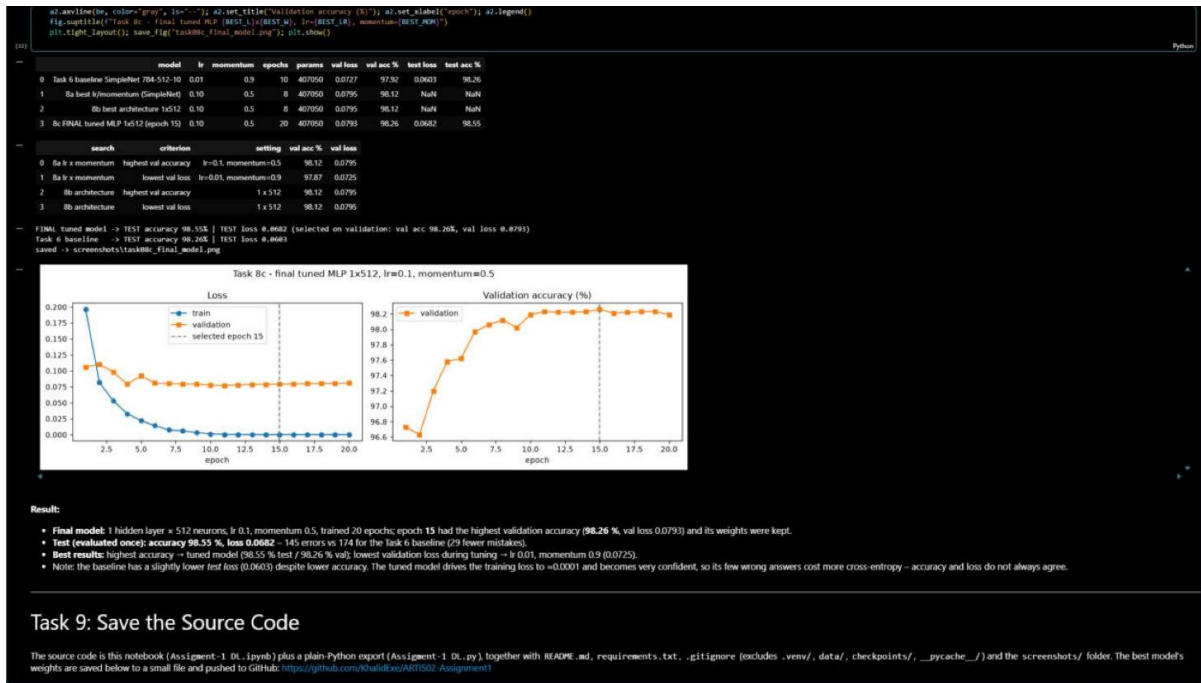


Figure 23: Source code output with the best results: summary tables and the single test evaluation

Model	Val loss	Val acc	Test loss	Test acc
Task 6 baseline (lr 0.01, momentum 0.9, 10 epochs)	0.0727	97.92 %	0.0603	98.26 %
Tuned MLP 1×512 (lr 0.1, momentum 0.5, epoch 15 of 20)	0.0793	98.26 %	0.0682	98.55 %

Result: epoch 15 had the highest validation accuracy (**98.26 %**, val loss 0.0793) and its weights were kept. On the test set this tuned model reaches **98.55 % accuracy (loss 0.0682)** – 145 errors instead of 174 for the baseline, with the same number of parameters. **Best results:** highest accuracy → tuned model (98.55 % test / 98.26 % validation); lowest validation loss during tuning → lr 0.01 with momentum 0.9 (0.0725). The baseline has a slightly lower test loss (0.0603) despite its lower accuracy: the tuned model drives its training loss to ≈ 0.0001 and becomes very confident, so its few wrong answers cost more cross-entropy – accuracy and loss do not always agree.

Task 9: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

Our Work & Results

• **Best results:** highest accuracy → tuned model (98.55 % test / 98.26 % val; lowest validation loss during tuning → lr 0.01, momentum 0.9 (0.0725).
• Note: the baseline has a slightly lower test loss (0.0603) despite lower accuracy. The tuned model drives the training loss to ≈ 0.0001 and becomes very confident, so its few wrong answers cost more cross-entropy – accuracy and loss do not always agree.

Task 9: Save the Source Code

The source code is this notebook (Assignment-1 DL.ipynb) plus a plain-Python export (Assignment-1 DL.py), together with README.md, requirements.txt, .gitignore (excludes .view/, data/, checkpoints/, __pycache__/) and the screenshots/ folder. The best model's weights are saved below to a small file and pushed to GitHub: <https://github.com/KhalidExe/ARTI502-Assignment1>

```
MODEL_PATH = os.path.join("models", "best_mlp.pt")
torch.save({"state_dict": final_model.state_dict(),
            "config": {"hidden_layers": BEST_L, "neurons": BEST_N, "lr": BEST_LR, "momentum": BEST_MOM,
                      "test_epoch": BE, "val_acc": final_val_acc, "test_acc": final_test_acc}, MODEL_PATH)
print("saved MODEL_PATH: (%s, %s) %s" % (MODEL_PATH, os.path.getsize(MODEL_PATH) / 1024, BE))

# reload check
ck = torch.load(MODEL_PATH, map_location=DEVICE)
reloaded = MLP(ck["config"]["hidden_layers"], ck["config"]["neurons"]).to(DEVICE)
reloaded.load_state_dict(ck["state_dict"])
print("reloaded model test accuracy: %.2f" % evaluate(reloaded, test_loader)[1])
```

— saved models/best_mlp.pt (1592.7 KB)
reloaded model test accuracy: 98.55%

Result:
The best model is saved to models/best_mlp.pt (1592.7 KB) together with its configuration. Reloading the file gives the same 98.55 % test accuracy, so the saved weights are valid. The source code is on GitHub.

Beyond Requirements (extra work – not part of the graded baseline)

Everything above follows the assignment exactly. The following experiments are **additional**.

B1) MLP: effect of Dropout and BatchNorm

Architecture 2 hidden layers × 512, SGD (lr 0.01, momentum 0.9), batch 32, 15 epochs. test accuracy is reported only, never used for selection.

```
RESULTS = [] # rows for the final comparison table
RESULTS.append(("model": "baseline (task 0)", "params": count_params(net), "epoch": EPOCHS,
               ["time s": BASE_TIME_S, "val acc %": BASE_VAL_ACC], [], "test acc %": TEST_ACC))
RESULTS.append(("model": "Tuned MLP (BEST_L)(BEST_N) (task 0)", "params": count_params(final_net), "epoch": FINAL_EPOCHS,
               ["time s": FINAL_TIME_S, "val acc %": FINAL_VAL_ACC, "test acc %": final_test_acc])

variables = {"alpha": dict(), "dropout 0.2": dict(dropout=0.2),
            "batchnorm": dict(batchnorm=True), "batchnorm + dropout 0.2": dict(batchnorm=True, dropout=0.2)}
reg_params, reg_hist = [], []
for name, kv in variables.items():
    n = MLP(2, 512, **kv)
    ll = make_loader(n)
```

Figure 24: Saving the best model weights and reloading them

The best model is saved to models/best_mlp.pt (1,592.7 KB) together with its configuration; reloading the file gives the same **98.55 %** test accuracy. The complete source code is saved and pushed to GitHub:

<https://github.com/KhalidExe/ARTI502-Assignment1>

File / folder	Content
Assignment-1 DL.ipynb	Main notebook – all tasks, executed with outputs
Assignment-1 DL.py	Plain-Python export of the notebook
screenshots/ + INDEX.md	All figures and VS Code screenshots, mapped to the task



File / folder	Content
	deliverables
models/best_mlp.pt	Weights of the best tuned MLP (Task 8c)
README.md, requirements.txt, .gitignore	Description and results, dependencies, ignored files (.env, data, checkpoints, __pycache__)
report/	This document (PDF and Word)

Beyond Requirements (extra work)

Everything above follows the assignment exactly. The experiments in this section are **additional** and were not used to change any required result.

B1) Effect of Dropout and BatchNorm on the MLP



Figure 25: MLP 2×512 with and without Dropout / BatchNorm (15 epochs)

All four variants reach ≈ 98.1 – 98.3 % validation accuracy. **BatchNorm** gives the lowest validation loss (0.0636 vs 0.0897 without) and the highest validation accuracy (98.27 %). **Dropout** reduces overfitting: the train–validation accuracy gap shrinks from 1.91 (plain) to 1.51 (dropout) and 1.03 (BatchNorm + dropout). Test accuracy differences are small (98.22–98.51 %).

B2) CNN with BatchNorm, Dropout, data augmentation, LR scheduler and early stopping

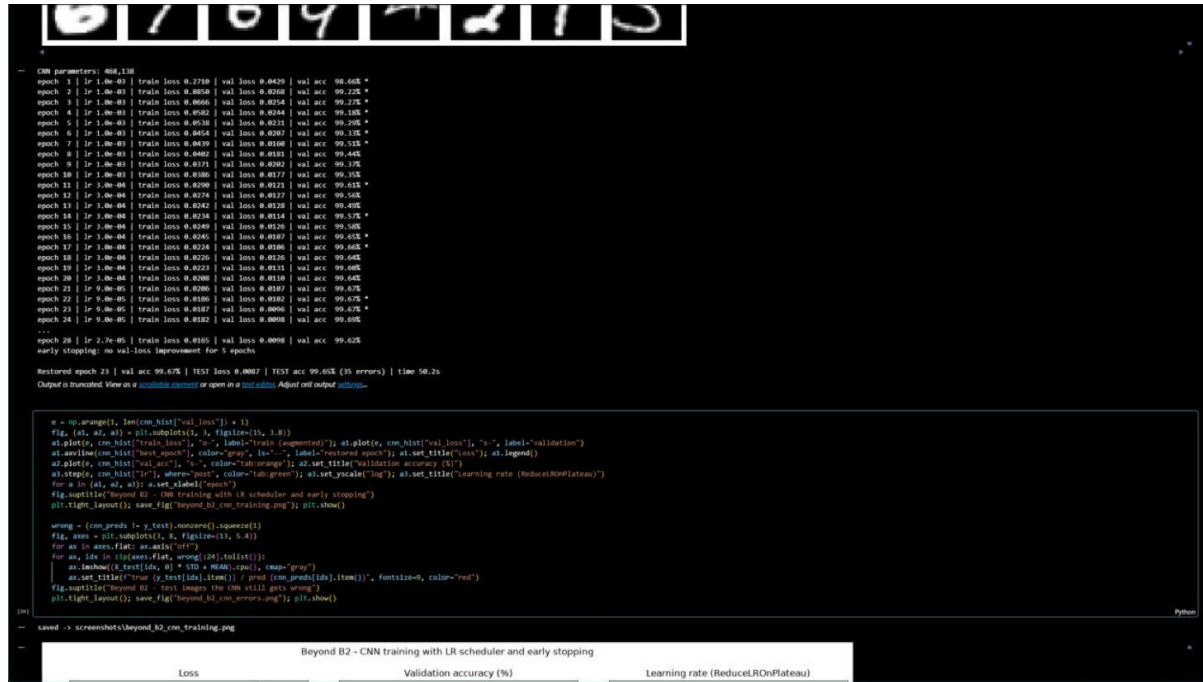


Figure 26: CNN training log – ReduceLRonPlateau and early stopping on validation loss

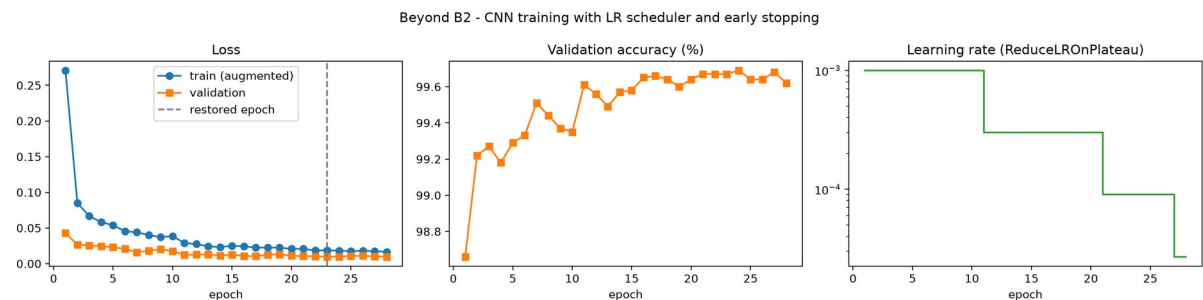


Figure 27: CNN loss, validation accuracy and learning-rate schedule

The CNN (4 conv layers with BatchNorm, 2 max-pool stages, Dropout; 468,138 parameters) is trained with light augmentation (rotation $\pm 10^\circ$, shift ± 2 px, scale 0.9–1.1) and Adam. ReduceLRonPlateau lowered the learning rate $1e-3 \rightarrow 3e-4 \rightarrow 9e-5 \rightarrow 2.7e-5$ and early stopping ended training after epoch 28; the weights of epoch 23 (lowest validation loss) were restored. Result: **validation 99.67 %**, **test accuracy 99.65 % (test loss 0.0087)** – only **35 errors** out of 10,000.



B3) CPU vs GPU training time

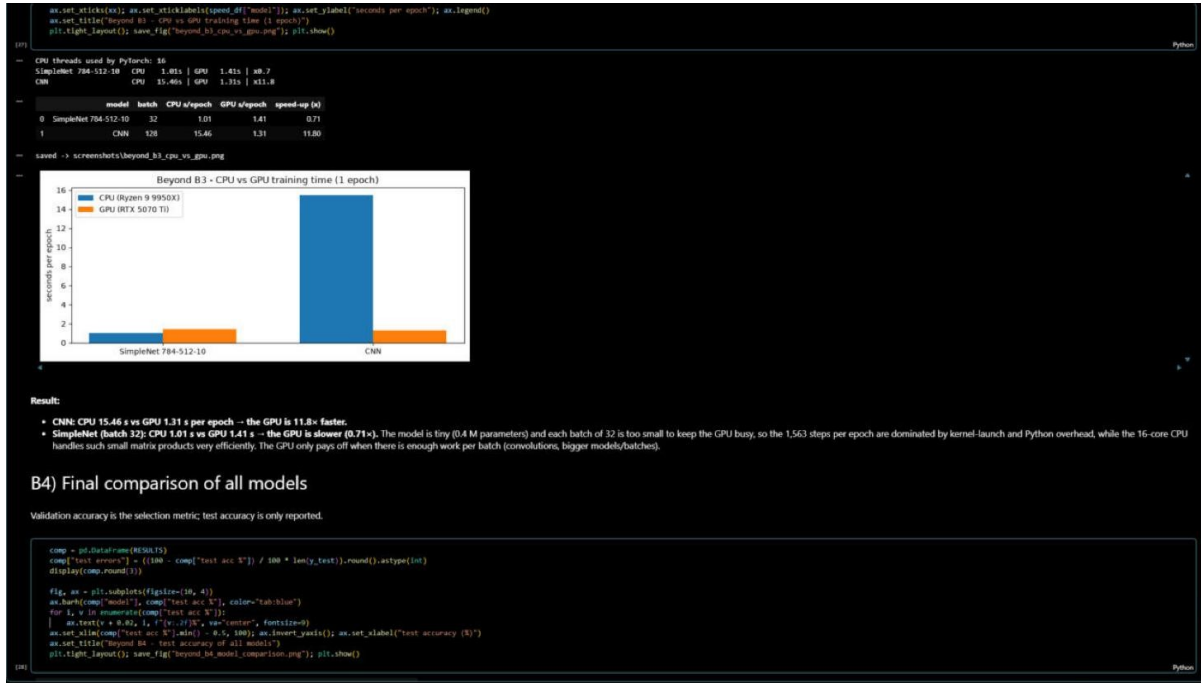


Figure 28: One training epoch on the CPU (Ryzen 9 9950X) and on the GPU (RTX 5070 Ti)

For the CNN the GPU is **11.8× faster** (15.46 s vs 1.31 s per epoch). For the small SimpleNet with batch 32 the GPU is actually slower (1.41 s vs 1.01 s): each batch is too small to keep the GPU busy, so kernel-launch and Python overhead dominate, while the 16-core CPU handles such small matrix products efficiently.

B4) Comparison of all models

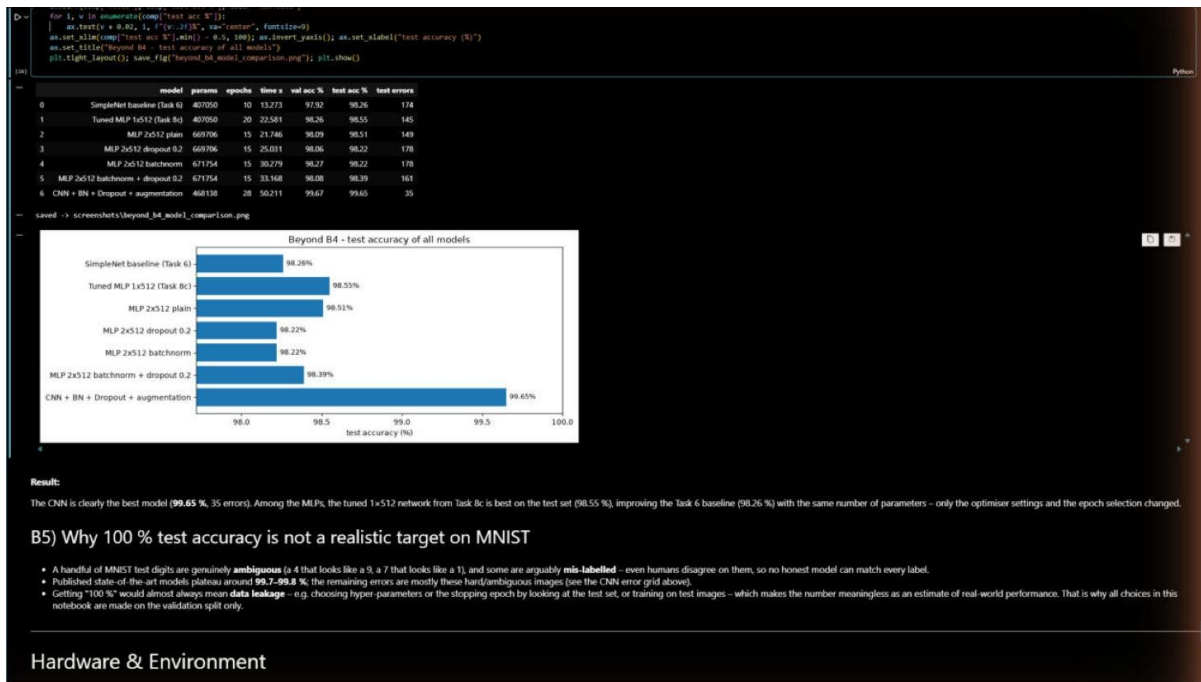


Figure 29: Final comparison of all models and why 100 % is not a realistic target

Model	Params	Epochs	Time (s)	Val acc	Test acc
Task 6 baseline MLP	407,050	10	13.3	97.92 %	98.26 %
Task 8c tuned MLP 1×512	407,050	20	22.6	98.26 %	98.55 %
MLP 2×512 plain	669,706	15	21.7	98.09 %	98.51 %
MLP 2×512 + dropout 0.2	669,706	15	25.0	98.06 %	98.22 %
MLP 2×512 + BatchNorm	671,754	15	30.3	98.27 %	98.22 %
MLP 2×512 + BatchNorm + dropout	671,754	15	33.2	98.08 %	98.39 %
CNN + BN + Dropout + augmentation	468,138	28	50.2	99.67 %	99.65 %

Why not 100 %? A few MNIST test digits are genuinely ambiguous (a 4 that looks like a 9, a 6 that looks like a 0) and some are arguably mislabelled, so even humans disagree on them. The best published models stop around 99.7–99.8 %. A “100 %” result would almost always mean **data leakage** – e.g. choosing hyper-parameters or the stopping epoch by looking at the test set – which is why all choices here were made on the validation set only.

Hardware & Environment

Component	Specification
CPU	AMD Ryzen 9 9950X (16 cores / 32 threads)
GPU	NVIDIA GeForce RTX 5070 Ti (Blackwell, 16 GB, compute capability 12.0)
RAM	64 GB DDR5 (61.7 GB usable)
OS / software	Windows 11, Python 3.14.4, PyTorch 2.14.0+cu130, CUDA 13.0, cuDNN 9.24, VS Code

Hardware & Environment

Component	Specification
CPU	AMD Ryzen 9 9950X (16 cores / 32 threads)
GPU	NVIDIA GeForce RTX 5070 Ti (Blackwell, 16 GB GDDR7)
RAM	64 GB DDR5
OS	Windows (see detected values below)

The next cell reads the values directly from the machine.

```

import psutil
def cpu_name():
    if platform.system() == "Windows":
        try:
            import wmi
            w = wmi.WMI()
            return wmi.WMI().Win32_Processor[0].Name
        except:
            return platform.processor()
    else:
        return platform.processor()

props = torch.cuda.get_device_properties(0)
env = {
    "OS": platform.platform(),
    "Python": sys.version.split()[0],
    "PyTorch": torch.__version__,
    "torchvision": torchvision.__version__,
    "CUDA (PyTorch build)": torch.version.cuda,
    "cuDNN": torch.backends.cudnn.version(),
    "GPU": props.name,
    "GPU VRAM": f"{props.total_memory / 1024**3:.1f} GB",
    "GPU compute capability": f"{props.major}.{props.minor}",
    "CPU": cpu_name(),
    "CPU cores / threads": f"{psutil.cpu_count(logical=False)} / {psutil.cpu_count(logical=True)}",
    "Total RAM": f"{psutil.virtual_memory().total / 1024**3:.1f} GB",
}
display(pd.DataFrame(env.items(), columns=["item", "value"]))
  
```

Item	Value
OS	Windows-11-10.0.26200-SP0
Python	3.14.4
PyTorch	2.14.0+cu130
torchvision	0.20.0+cu130
CUDA (PyTorch build)	13.0
cuDNN	9.0.400
GPU	NVIDIA GeForce RTX 5070 Ti
GPU VRAM	15.9 GB
GPU compute capability	12.0

Figure 30: Hardware and software detected automatically by the notebook

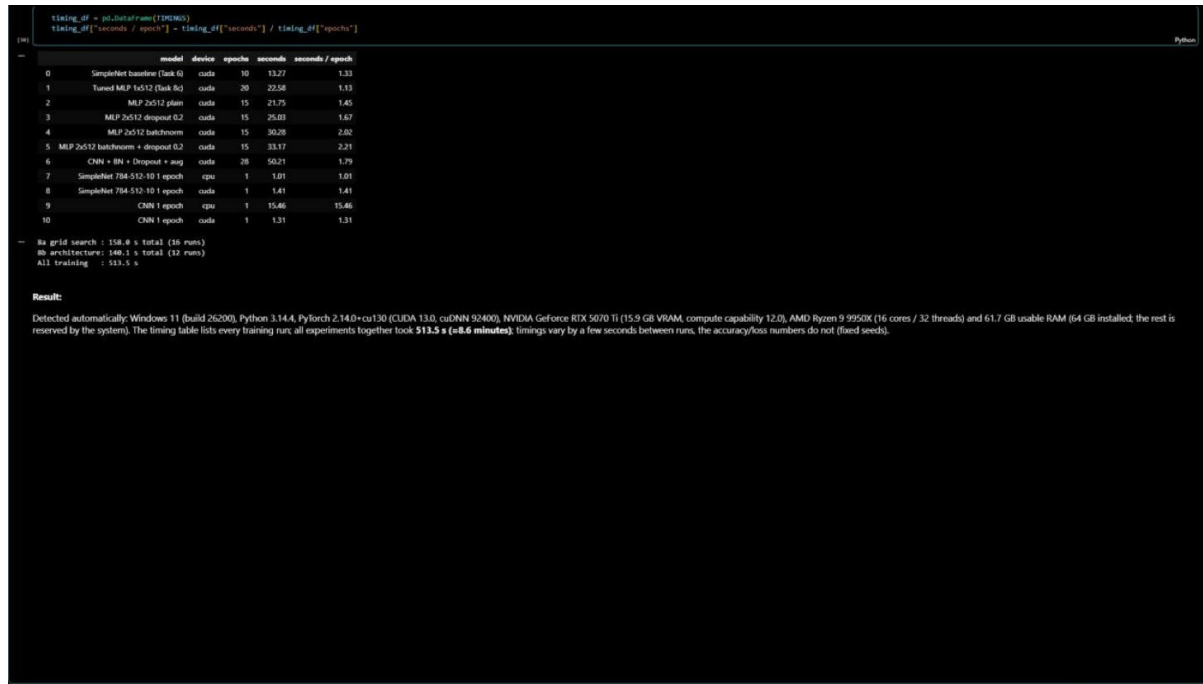


Figure 31: Training time of every model

All training runs together took 513.5 s (~8.6 min): 158.0 s for the 16 learning-rate/momentum runs and 140.1 s for the 12 architecture runs. Times vary by a few seconds between runs; accuracy and loss values do not (fixed seeds).



- End of Assignment - Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Install PyTorch	PyTorch installed successfully and PyTorch loaded correctly with clear structure	Installed and PyTorch loaded with minor issues	pyTorch exists but unclear or incomplete	Not completed / missing	0.5
Task 2: Download and Load the Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	0.5
Task 3: Inspect the Dataset	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	0.5
Task 4: Creating A Simple Network	A simple network correctly created, screenshot included	Minor reporting issues	Node used but missing result explanation or screenshot	Not done	2
Task 6: Training Function	Training function configured correctly with clear screenshots of output	Minor reporting issues	Training Function used but missing result explanation or screenshot	Not done	1
Task 7: Testing Network	Correct Testing network created, screenshot included.	Testing network correct but result description is weak	Testing network exists but unclear results description	Not done	1
Task 8: Tuning the parameters and model architecture	Tuning parameters and model architecture created, screenshot included, and results description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture correct but incorrect results description	Not done	1.5
Task 9: Save your source code	Source code save successfully	Source code save successfully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	0.5
Total					7.5